WTBs2025 limpio a 2 clases

Cómo se preparó el dataset y qué hay que hacer para combinarlo con el dataset DTU de Kaggle · Proyecto Final, inspección de palas con dron · 23/10/2026

Resumen

El archivo **WTBs2025_2cls.zip** contiene las 7 544 imágenes del dataset WTBs2025 corregidas a 640×480, con los labels sin duplicados, re-etiquetadas a dos clases (**0 = damage**, **1 = dirt**) y ya divididas en train / val / test sin fuga de datos. Todas las cajas de este dataset son **damage**: no tiene ninguna de **dirt**, que va a salir del dataset DTU.

Tres reglas para no arruinar el experimento:

1. El test que decide la compra de la cámara se arma **solo con DTU** re-muestreado a 5 mm/px. WTBs2025 no sirve para eso porque su GSD es desconocido.
2. El split del DTU se hace **por foto original** (prefijo `DJI_XXXX`), nunca por recorte.
3. Antes de combinar, verificar que en el DTU **damage = 0 y dirt = 1**.

1. Origen del dataset

WTBs2025 es un dataset público publicado en *Scientific Data* (Zhang et al., 2026). Son fotos de palas tomadas en inspecciones de rutina en varios parques eólicos de China (Shanxi, Gansu y Shaanxi) entre 2022 y 2024, con cámaras RGB de 4000×3000 px. Los autores redimensionaron todas las imágenes a 640×640 y las anotaron en formato YOLO con 9 clases de falla. Las características que importan para este proyecto son:

- **Muchas turbinas y fondos distintos.** Complementa al DTU, que es una sola turbina.
- **GSD desconocido y variable.** Hay primeros planos y palas enteras con paisaje, y no se publicaron los originales de 4000×3000.
- **Todas las imágenes tienen al menos una falla.** No hay fotos de pala limpia.
- **Lightning strike está aumentada.** Son 57 fotos reales, expandidas a 228 con rotación, brillo y espejado, más 57 imágenes generadas con una GAN (GA-DCGAN): 285 en total.

2. Qué se le hizo

Se usaron dos scripts propios, `split_wtbs2025.py` y `make_2class.py`. Los pasos, en orden:

Paso	Qué se hizo	Por qué
1. Aspecto 4:3	640×640 → 640×480 (LANCZOS). Las coordenadas YOLO normalizadas no cambian. En una muestra de 200 imágenes no hubo bandas negras (0 % letterbox).	Los originales eran 4000×3000. El 640×640 las había deformado (las fallas quedaban más angostas).
2. Cajas duplicadas	Se eliminaron 1 604 líneas repetidas en los labels.	Una caja repetida en val/test cuenta como falso negativo y baja el mAP.
3. Agrupado anti-fuga	pHash de 64 bits (original, espejo H y V) en todo el dataset, más ORB + RANSAC con transformación de similitud en lightning strikes (copias rotadas). Resultado: 7 544 imágenes en 5 882 grupos.	Una foto y sus copias aumentadas o casi idénticas tienen que caer en el mismo conjunto. Si no, el test mide memoria, no detección.
4. Split 70/15/15	Por grupo, estratificado por carpeta de clase, semilla 0.	Todas las clases aparecen en val y en test.
5. Sintéticas	Los grupos de 1 imagen en lightning strikes se marcaron como posibles GAN y se mandaron solo a train (4 imágenes).	No se evalúa con imágenes generadas. Ver la limitación de la sección 6.

Paso	Qué se hizo	Por qué
6. 2 clases	Las 9 clases → 0 = damage. La clase 1 = dirt queda vacía en este dataset. Surface stains se revisó visualmente y corresponde a daño.	Taxonomía común con el DTU (damage / dirt).

3. Resultado

Imágenes y cajas por conjunto (cajas únicas, después de quitar duplicados):

Conjunto	Imágenes	% imágenes	Cajas damage	Cajas dirt	Sintéticas
train	5 385	71,4 %	7 786	0	4
val	1 086	14,4 %	1 581	0	0
test	1 073	14,2 %	1 566	0	0
Total	7 544	100 %	10 933	0	4

Cajas por clase original (todas pasan a damage). Sirve para ver si el reparto quedó parejo y para analizar después qué tipos de falla detecta peor el modelo:

Clase original	train	val	test	Total
erosion	3 207	687	624	4 518
coating detachment	1 373	267	266	1 906
localized damage	900	180	174	1 254
pinholes	635	115	139	889
protective film damage	448	92	92	632
oil leakage	413	80	79	572
paint cracks	379	65	70	514
lightning strikes	222	53	79	354
surface stains	209	42	43	294
Total	7 786	1 581	1 566	10 933

4. Contenido del ZIP

```
WTBs2025_2cls/
├── data.yaml              # train/val/test y names {0: damage, 1: dirt}; sin 'path'
├── images/
│   ├── train/   (5 385)
│   ├── val/     (1 086)
│   └── test/    (1 073)
├── labels/                # mismo nombre que la imagen, .txt en formato YOLO
│   ├── train/  val/  test/
└── manifest.csv           # image, split, orig_folder, orig_classes, synthetic_candidate
```

- **Formato de cada línea de label:** `clase x_centro y_centro ancho alto`, normalizados a [0, 1] respecto de la imagen.
- **data.yaml sin 'path':** Ultralytics toma la carpeta del propio archivo, así funciona igual en local y en `/kaggle/input/...`. Si se queja, agregar `path:` con la ruta del dataset en Kaggle.
- **manifest.csv:** guarda la clase original de cada imagen. Con eso se calcula el recall del modelo de 2 clases por tipo de falla (por ejemplo, si se le escapan los pinholes).

5. Cómo combinarlo con el DTU (pasos para Kaggle)

El dataset DTU de Kaggle (*YOLO Annotated Wind Turbine Surface Damage*, 586×371 px) son recortes de fotos de dron de 5280×2970 px de una única turbina (Nordtank, DTU, Roskilde), con dos clases: damage y dirt. Tiene unas 13 470 imágenes, y menos del 25 % contiene algún defecto.

5.1 Subir y verificar

1. Subir `WTBs2025_2cls.zip` como Kaggle Dataset **privado**. `/kaggle/input` es de solo lectura, así que todo lo procesado (gris, ruido, tiles) se escribe en `/kaggle/working`.
2. **Verificar los IDs de clase del DTU** (su classes.txt o data.yaml). Las fuentes dicen "Dirt and Damage" sin indicar el orden. Si es `0 = dirt, 1 = damage`, hay que re-mapear uno de los dos datasets antes de combinarlos.

5.2 Split del DTU por foto original

Los recortes se llaman, por ejemplo, `DJI_0201_02_06.png`: `DJI_0201` es la foto original y los dos números son la posición del recorte. Hay que hacer el split **por el prefijo DJI_XXXX**. Si recortes de una misma foto caen en train y en test, el mAP sale inflado. Con 5280/586 ≈ 9 y 2970/371 ≈ 8, cada foto daría una grilla de unos 9 × 8 recortes (es una inferencia: confirmar cuál índice es fila y cuál columna). Hay que tener en cuenta que las ~13 000 imágenes salen de solo unos cientos de fotos.

5.3 Qué procesamiento lleva cada dataset

Etapa	DTU	WTBs2025_2cls
Re-muestreo a 5 mm/px	Sí: s = GSD_orig / 5 mm/px, cv2.INTER_AREA. Hay que estimar el GSD (objeto de tamaño conocido en la pala, o GSD = d·p/f).	**No**: el GSD es desconocido. Se usa tal cual.
Escala de grises (rama B)	Sí	Sí
Ruido de sensor + blur leve	Sí	Sí
Tiling 2×2	Sí, sobre el frame equivalente del OV9281 (ver 5.4)	**No**: cada imagen de 640×480 ya equivale a un tile. Se lleva directo a la entrada de 320 px.
Filtrar cajas chicas	Sí: descartar cajas con lado < 3 px a la entrada del modelo (320 px)	Sí, mismo criterio
Balance de fondo	Quedarse con ~10–20 % de tiles sin cajas	No aplica (no hay imágenes limpias)
Test de decisión	**Sí**	No (se reporta aparte, como referencia)

Las ramas A (RGB) y B (gris) usan exactamente el mismo split, el mismo modelo y los mismos hiperparámetros. Lo único que cambia es la imagen de entrada. Las imágenes en gris se guardan con un solo canal: Ultralytics las lee replicándolo a 3 canales, así se aprovechan los pesos preentrenados.

5.4 Tiling con solape y recálculo de cajas

Con un frame de W×H y dos tiles por eje con solape fraccional *o*, el tamaño de cada tile es t_w = W / (2 − o) y t_h = H / (2 − o). Los orígenes quedan en $x_0 \in \{0, W - t_w\}$ e $y_0 \in \{0, H - t_h\}$. Para 1280×720: con o = 10 % el tile es de 674×379 px, con 15 % de 692×389 px y con 20 % de 711×400 px. Ojo: tiles de exactamente 640×360 cubren el frame **sin** solape. Cada caja se recorta al tile y se re-normaliza; se descarta si queda visible menos del 40 % o si su lado es muy chico:

```
def boxes_in_tile(boxes, W, H, x0, y0, tw, th, min_vis=0.4, min_px=6):
    """boxes: [(c, xc, yc, w, h)] normalizados a la imagen completa W x H.
    min_px en píxeles del tile (6 px en un tile de ~640 ≈ 3 px a la entrada de 320)."""
    out = []
    for c, xc, yc, w, h in boxes:
        x1, y1 = (xc - w / 2) * W, (yc - h / 2) * H
        x2, y2 = (xc + w / 2) * W, (yc + h / 2) * H
        ix1, iy1 = max(x1, x0), max(y1, y0)
        ix2, iy2 = min(x2, x0 + tw), min(y2, y0 + th)
        if ix2 <= ix1 or iy2 <= iy1:
            continue                                    # no toca el tile
        vis = (ix2 - ix1) * (iy2 - iy1) / ((x2 - x1) * (y2 - y1))
        if vis < min_vis or min(ix2 - ix1, iy2 - iy1) < min_px:
            continue                                    # muy cortada o muy chica
        out.append((c, ((ix1 + ix2) / 2 - x0) / tw, ((iy1 + iy2) / 2 - y0) / th,
                    (ix2 - ix1) / tw, (iy2 - iy1) / th))
    return out
```

5.5 YAML combinado

Una carpeta por rama (`combined_rgb` y `combined_gray`), cada una con el mismo esquema:

```
# /kaggle/working/combined_gray/data.yaml
path: /kaggle/working/combined_gray
train:
  - wtbs/images/train
  - dtu/images/train
val:
  - wtbs/images/val
  - dtu/images/val
test: dtu/images/test          # test de decisión: solo DTU a 5 mm/px
names:
  0: damage
  1: dirt
```

5.6 Entrenar, cuantizar y evaluar

```
from ultralytics import YOLO

m = YOLO("yolov8n.pt")
m.train(data="/kaggle/working/combined_gray/data.yaml", imgsz=320, seed=0)
m.val(data="/kaggle/working/combined_gray/data.yaml", split="test")   # mAP@0.5 por clase

# int8 con imágenes representativas del dataset; después edgetpu_compiler
m.export(format="tflite", int8=True, imgsz=320,
         data="/kaggle/working/combined_gray/data.yaml")
```

- La cantidad de épocas y el resto de los hiperparámetros quedan a criterio, pero tienen que ser **iguales en las dos ramas**.
- El mAP que vale para la decisión es el del **modelo int8** sobre el test DTU. La cuantización puede bajar la precisión respecto del modelo en float.
- Revisar los argumentos de exportación en la documentación de Ultralytics para la versión instalada. Al compilar con `edgetpu_compiler`, verificar en el log que todas las operaciones queden en la TPU.
- Además, reportar aparte el mAP sobre el test de WTBs2025. Con `manifest.csv` se puede calcular el recall por clase original.

6. Limitaciones y avisos

- **Imágenes GAN.** Se esperaban unas 57 sueltas y se detectaron 4. Lo más probable es que cada imagen generada haya quedado agrupada con la foto real en la que se basó (los grupos de lightning strikes son de 5 o 6 en vez de 4). En ese caso no hay fuga, pero puede haber algunas sintéticas en val o test de lightning strikes. Para el modelo de 2 clases el efecto es mínimo: lightning strikes es el 3,2 % de las cajas.
- **Grupo de 256 imágenes.** El pHash unió en cadena muchas fotos parecidas (oil leakage: 520 imágenes en 31 grupos). Agrupar de más no genera fuga: solo puede desbalancear el reparto, y el reparto quedó parejo.
- **Sin dirt ni imágenes limpias.** Si alguna foto de WTBs2025 tiene suciedad, el modelo la aprende como fondo. Los negativos (pala sin defectos) salen del DTU.
- **Corrección de aspecto.** Supone que el 640×640 se hizo deformando (stretch) desde 4:3. Es consistente con el 0 % de letterbox medido.
- **Métricas del paper.** No sirven como referencia: el split fue aleatorio con copias aumentadas adentro, y entrenaron a 640 px en color.

7. Referencias y licencias

- Zhang, R. et al. *Wind turbine blades surface defect high-quality image dataset construction and performance validation.* Scientific Data 13:1310 (2026). doi:10.1038/s41597-026-07455-1. Artículo CC BY-NC-ND 4.0; el dataset se publica para uso académico y de investigación.
- Dataset WTBs2025 en figshare: doi:10.6084/m9.figshare.28876391.
- Foster, A. *YOLO Annotated Wind Turbine Surface Damage*. Mendeley Data V1 (2021). doi:10.17632/t6fwpc735s.1. CC BY-NC 4.0.
- Shihavuddin, A. S. M. & Chen, X. *DTU – Drone inspection images of wind turbine*. Mendeley Data V2 (2018). doi:10.17632/hd96prn3nc.2.